동적 스코어링 수학 모델 확장: 방법론 및 구현 상세 보고서

본 보고서는 KOSPI 200 종목을 대상으로 설계된 3-Layer 스윙 트레이딩 시스템의 고정 가중치 모델을 동적 가중치 스코어링 수학 모델로 확장하기 위한 상세 방법론을 제시한다. 가중치가 시장 상태(변동성, 팩터 유효성)에 따라 매일 자동으로 조정되는 "동적" 특성을 구현하며, Python 환경 제약 및 일봉 데이터 사용 가능 조건을 준수한다.

1. 동적 가중치 산출 공식 및 방법론 비교

팩터의 동적 가중치는 시장 환경 변화에 따라 팩터의 유효성이 달라진다는 전제 하에, 각 시점 t에서 개별 팩터 i의 가중치 $w_i(t)$를 조정하는 핵심 요소이다. 최종 동적 스코어 $Score_t$는 각 시점 t에서 산출된 동적 가중치와 정규화된 팩터 점수의 가중 합으로 계산된다.

최종 동적 스코어 산출 공식: $$ Score_t = \sum_{i=1}^{N} w_i(t) \times \text{Normalize}(Factor_i(t)) \times 100 $$ 여기서, N은 전체 팩터의 수, $Factor_i(t)$는 시점 t에서의 i번째 팩터 점수이며, $\text{Normalize}(\cdot)$는 팩터 점수를 0과 1 사이로 정규화하는 함수이다.

1.1. IC 가중 (Information Coefficient Weighting)

정보 계수(IC) 가중 방식은 각 팩터의 예측력에 비례하여 가중치를 할당하는 직관적인 방법이다. 팩터 i의 시점 t 가중치 $w_i(t)$는 해당 팩터의 Information Coefficient (IC) 절댓값에 비례하여 산출된다. IC는 팩터 값의 순위와 실제 수익률 순위 간의 스피어만 순위 상관 계수로 정의되며, 롤링 윈도우(예: 20일)를 사용하여 계산하여 최근 시장 데이터를 반영한다.

IC 가중치 산출 공식: $$ w_i(t) = \frac{|IC_i(t)|}{\sum_{j=1}^{N} |IC_j(t)|} $$ 여기서, $IC_i(t)$는 시점 t에서 계산된 i번째 팩터의 정보 계수이다. 롤링 20일 기간 동안의 팩터 값과 미래 수익률 간의 상관관계를 통해 계산된다.

Python 수도코드:

```python
import pandas as pd
import numpy as np

def calculate_ic(factor_values, future_returns, window=20):
    """
    팩터 값과 미래 수익률 간의 롤링 IC를 계산합니다.
    Args:
        factor_values (pd.Series): 각 시점의 팩터 값.
        future_returns (pd.Series): 각 시점의 미래 수익률.
        window (int): 롤링 윈도우 크기.
    Returns:
        pd.Series: 각 시점의 롤링 IC 값.
    """
    # 순위 기반 상관계수 (Spearman's rank correlation) 계산
    ic_series =
factor_values.rolling(window=window).corr(future_returns.rolling(wind
ow=window).rank(pct=True))
    return ic_series

def ic_weighting(ics):
    """
    계산된 IC 값을 바탕으로 가중치를 산출합니다.
    Args:
        ics (dict): {factor_name: pd.Series(ic_values)} 형태의 IC 값 딕
셔너리.
    Returns:
        pd.DataFrame: 각 시점별 팩터 가중치.
    """
    abs_ics = pd.DataFrame({name: ic.abs() for name, ic in
ics.items()})
    sum_abs_ics = abs_ics.sum(axis=1)
    # 0으로 나누는 오류 방지
    weights = abs_ics.divide(sum_abs_ics, axis=0).fillna(0)
    return weights

# 예시
# factor1_ic = calculate_ic(df, df)
# factor2_ic = calculate_ic(df, df)
# factor_ics = {'Factor1': factor1_ic, 'Factor2': factor2_ic}
# dynamic_weights_ic = ic_weighting(factor_ics)
```

1.2. 시장 레짐 스위칭 (Market Regime Switching)

시장 레짐 스위칭은 시장의 전반적인 상태(예: 강세장, 약세장, 횡보장)를 식별하고, 각 레짐에 따라 미리 정의된 가중치를 적용하는 방법이다. 금융 시장은 상승과 하락이 교차하는 기간을 보이며, 트레이더는 이러한 시장 추세의 전환을 정확하게 예측하여 투자 결정을 내려야 한다. KOSPI 변동성과 모멘텀을 사용하여 시장 레짐을 정의할 수 있다. 예를 들어, KOSPI 변동성이 낮고 모멘텀이 양수이면 강세장으로, 변동성이 높고 모멘텀이 음수이면 약세장으로 분류하는 식이다. Hidden Markov

Model (HMM)은 금융 시계열 데이터에서 숨겨진 시장 상태를 모델링하는 데 효과적인 도구이다. HMM은 시장의 다양한 레짐 간 전환을 명시적으로 모델링하며, 시간 가변 파라미터를 사용한 HMM은 일일 수익률의 장기 기억 효과를 재현하고 예측 정확도를 향상시킬 수 있다.

시장 레짐 정의 함수: $$ \text{regime}_t = f(\text{KOSPI_volatility}_t, \text{KOSPI_momentum}_t) $$ 각 레짐에 대한 가중치 매핑: $w_i(t) = \text{WeightMap}(\text{regime}_t)$.

Python 수도코드:

```python
from sklearn.mixture import GaussianMixture # HMM 대신 GMM을 활용한 간단
한 레짐 분류 예시

def identify_market_regime(kospi_volatility, kospi_momentum,
n_regimes=3):
    """
    KOSPI 변동성과 모멘텀을 사용하여 시장 레짐을 식별합니다.
    Args:
        kospi_volatility (pd.Series): KOSPI 지수 변동성.
        kospi_momentum (pd.Series): KOSPI 지수 모멘텀.
        n_regimes (int): 식별할 시장 레짐의 수 (예: 3 = 강세, 횡보, 약세).
    Returns:
        pd.Series: 각 시점의 시장 레짐 인덱스.
    """
    # 특징 데이터를 결합
    features = pd.DataFrame({'volatility': kospi_volatility,
'momentum': kospi_momentum}).dropna()

    # GMM을 사용하여 레짐 분류 (간단한 예시, 실제 HMM은 더 복잡)
    gmm = GaussianMixture(n_components=n_regimes, random_state=42)
    gmm.fit(features)
    regimes = pd.Series(gmm.predict(features), index=features.index)
    return regimes

def map_regime_weights(regime, regime_weight_map):
    """
    식별된 레짐에 따라 팩터 가중치를 매핑합니다.
    Args:
        regime (int): 현재 시점의 시장 레짐 인덱스.
        regime_weight_map (dict): {regime_idx: {factor_name: weight}}
형태의 매핑.
    Returns:
        dict: 현재 레짐에 해당하는 팩터 가중치.
    """
    return regime_weight_map.get(regime, {})

# 예시
# kospi_vol = df
# kospi_mom = df
# market_regimes = identify_market_regime(kospi_vol, kospi_mom)
# # 레짐별 가중치 사전 정의 (백테스팅을 통해 최적화 필요)
# pre_defined_weights = {
#     0: {'Factor1': 0.6, 'Factor2': 0.3, 'Factor3': 0.1}, # Regime 0
(e.g., Bull Market)
#     1: {'Factor1': 0.3, 'Factor2': 0.4, 'Factor3': 0.3}, # Regime 1
(e.g., Sideways Market)
#     2: {'Factor1': 0.2, 'Factor2': 0.2, 'Factor3': 0.6}, # Regime 2
(e.g., Bear Market)
# }
```

```
# dynamic_weights_regime = market_regimes.apply(lambda r:
map_regime_weights(r, pre_defined_weights))
```

1.3. ML 메타모델 (XGBoost를 활용한 최적 가중치 예측)

ML 메타모델은 팩터의 유효성, 시장 변동성, 모멘텀 등의 상태 변수를 입력으로 받아 각 팩터의 최적 가중치를 직접 예측하는 고급 방법론이다. XGBoost는 트리 부스팅(tree boosting) 기반의 효과적인 머신러닝 기법으로, 동적 가중치 다중 팩터 주식 선정 전략에 활용될 수 있다. 특히, XGBoost 모델은 팩터의 IC 계수를 예측하는 데 효과적이며, 이를 통해 동적 가중치 전략의 성능을 향상시킬 수 있음이 실증적으로 증명되었다. 이는 팩터의 예측력 변화를 머신러닝으로 학습하여 가중치에 반영하는 방식이다.

ML 메타모델 가중치 산출 공식: \[w_i(t) = \text{XGBoost}(\text{IC}_i(t-1), \text{volatility}(t-1), \text{momentum}(t-1), \dots) \] 여기서, XGBoost 모델은 과거 데이터를 학습하여 주어진 입력 변수들로부터 각 팩터의 최적 가중치를 출력한다.

Python 수도코드:

```python
import xgboost as xgb
from sklearn.model_selection import train_test_split

def train_ml_meta_model(features, target_weights):
    """
    XGBoost 메타모델을 훈련하여 팩터 가중치를 예측합니다.
    Args:
        features (pd.DataFrame): IC, 변동성, 모멘텀 등 입력 특징.
        target_weights (pd.DataFrame): 각 시점의 목표 팩터 가중치 (예: 사후
최적 가중치).
    Returns:
        dict: {factor_name: xgb.Booster} 형태의 팩터별 훈련된 모델.
    """
    models = {}
    for factor_name in target_weights.columns:
        X_train, X_test, y_train, y_test = train_test_split(
            features, target_weights, test_size=0.2, random_state=42
        )
        model = xgb.XGBRegressor(objective='reg:squarederror',
n_estimators=100, learning_rate=0.05, max_depth=5)
        model.fit(X_train, y_train, eval_set=,
early_stopping_rounds=10, verbose=False)
        models = model
    return models

def predict_ml_weights(trained_models, current_features):
    """
    훈련된 메타모델을 사용하여 현재 시점의 팩터 가중치를 예측합니다.
    Args:
        trained_models (dict): train_ml_meta_model에서 반환된 모델 딕셔너
리.
        current_features (pd.DataFrame): 현재 시점의 입력 특징.
    Returns:
        pd.Series: 예측된 팩터 가중치.
    """
    predicted_weights = {name: model.predict(current_features) for
name, model in trained_models.items()}
    # 가중치 합이 1이 되도록 정규화하고 음수 가중치 처리
    normalized_weights = np.maximum(0,
list(predicted_weights.values()))
    if normalized_weights.sum() > 0:
        normalized_weights = normalized_weights /
normalized_weights.sum()
    return pd.Series(normalized_weights,
index=predicted_weights.keys())

# 예시
# # target_weights는 백테스트를 통해 도출된 최적 가중치 또는 벤치마크 가중치로 가
정
# # features는 IC, KOSPI vol, KOSPI mom 등을 포함
# # trained_meta_models = train_ml_meta_model(features_data,
```

```
optimal_past_weights)
# # current_features_for_prediction = pd.DataFrame([],
columns=features_data.columns)
# # dynamic_weights_ml = predict_ml_weights(trained_meta_models,
current_features_for_prediction)
```

1.4. 동적 가중치 방법론 장단점 비교표

방법론	장점	단점	최적안 제시 이유
IC 가중	- 구현 용이성 및 직관성 - 팩터의 실시간 예측력 반영	- IC의 노이즈에 민감 - 단일 선형 관계에 초점, 비선형성 포착 어려움	시장 상황 변화에 팩터 유효성 변화를 반영하나, IC 자체의 불안정성이 가중치 변동성을 높일 수 있어 **보조적인 방법**으로 활용하는 것이 좋다.
시장 레짐 스위칭	- 시장 전반적인 상황 변화에 유연하게 대응 - 블랙 스완(Black Swan) 이벤트 방어	- 레짐 정의 및 전환 감지 정확도가 중요 - 레짐별 최적 가중치 사전 정의 필요	시장의 큰 흐름에 따른 가중치 변화는 유용하나, 레짐 식별의 정확도와 레짐 내에서의 세부 팩터 조정의 한계가 있다. ML 메타모델의 입력 특징으로 활용하여 보완 가능하다.
ML 메타모델 (XGBoost)	- 복잡한 비선형 관계 학습 가능 - 다양한 시장 변수 통합 고려 - IC 예측 개선	- 과적합 위험 - 해석의 어려움 - 충분한 학습 데이터 필요	ML 메타모델{{1}}은 팩터 유효성, 시장 변동성, 모멘텀 등 복합적인 시장 환경을 고려하여 각 팩터의 최적 가중치를 직접 예측하므로 가장 진보된 접근 방식이다. XGBoost는 특히 이러한 유형의 예측에서 우수한 성능을 보인다. 50,000 샘플 규모에서는 과적합 관리가 중요하지만, 강력한 성능 향상을 기대할 수 있는 **최적안**이다.

2. 정규화 로직 구체화

각 지표는 단위와 분포가 다르므로, 정규화{{2}}는 팩터 스코어를 의미 있는 방식으로 비교하고 결합하기 위해 필수적이다. 0-1 범위로 정규화하는 것은 모델의 안정성과 해석 가능성을 높인다.

2.1. Min-Max Scaling (최소-최대 스케일링)

안정적인 지표(PER, PBR 등)에 적합하며, 데이터의 최댓값과 최솟값을 사용하여 특정 범위(예: 0~1)로 선형적으로 스케일링한다. 극단값에 민감하게 반응하여 분포의 왜곡이 큰 경우에는 적합하지 않을 수 있다.

수학적 정의: $$ X_{normalized} = \frac{X - X_{min}}{X_{max} - X_{min}} $$ 여기서, X는 원본 데이터, X_{min}은 데이터의 최솟값, X_{max}는 데이터의 최댓값이다.

Python 수도코드:

```python
def min_max_scaling(series):
    """
    Min-Max Scaling을 적용하여 데이터를 0-1 범위로 정규화합니다.
    Args:
        series (pd.Series): 정규화할 데이터 시리즈.
    Returns:
        pd.Series: 정규화된 데이터 시리즈.
    """
    min_val = series.min()
    max_val = series.max()
    if max_val == min_val: # 분모가 0이 되는 경우 방지
        return pd.Series(0.5, index=series.index) # 또는 다른 처리 방식
    return (series - min_val) / (max_val - min_val)

# 예시
# df = min_max_scaling(df)
# df = min_max_scaling(df)
```

2.2. Sigmoid 정규화 (Sigmoid Normalization)

Sigmoid 함수는 극단값의 영향을 완화하며, 데이터를 0과 1 사이의 부드러운 S자 곡선 형태로 매핑한다. RSI, MACD와 같이 상하한이 존재하거나 분포가 한쪽으로 치우쳐 극단값이 자주 발생하는 지표에 적합하다.

수학적 정의: $$ \sigma(z) = \frac{1}{1 + e^{-z}} $$ 여기서, z는 정규화할 원본 값에 대한 변환값으로, 일반적으로 $z = \frac{X - \mu}{\sigma}$ (Z-score) 형태가 사용될 수 있다.

Python 수도코드:

```python
def sigmoid_normalization(series, center=0, scale=1):
    """
    Sigmoid 정규화를 적용합니다.
    Args:
        series (pd.Series): 정규화할 데이터 시리즈.
        center (float): 데이터의 중심을 맞출 값 (예: 평균).
        scale (float): 데이터의 스케일을 조절할 값 (예: 표준편차).
    Returns:
        pd.Series: 정규화된 데이터 시리즈.
    """
    z = (series - center) / scale
    return 1 / (1 + np.exp(-z))

# 예시
# # RSI는 0-100 범위이므로, 50을 중심으로 표준편차로 스케일링 가능
# df = sigmoid_normalization(df, center=50, scale=10) # scale은 조정 필요
# # MACD는 +/- 무한대이므로, 평균과 표준편차를 직접 사용
# df = sigmoid_normalization(df, center=df.mean(), scale=df.std())
```

2.3. Percentile Rank (백분위 순위)

유니버스 내 상대적 순위를 기반으로 정규화하므로 분포에 대한 가정이 불필요하며, 횡단면 비교에 특히 유용하다. 극단값에 강하며, 모든 데이터를 0과 1 사이의 균일한 분포로 변환한다.

수학적 정의: \[\text{PercentileRank}(X_i) = \frac{\text{Number of observations} \le X_i}{\text{Total number of observations}} \]

Python 수도코드:

```python
def percentile_rank_normalization(series):
    """
    Percentile Rank 정규화를 적용하여 데이터를 0-1 범위로 정규화합니다.
    Args:
        series (pd.Series): 정규화할 데이터 시리즈.
    Returns:
        pd.Series: 정규화된 데이터 시리즈.
    """
    return series.rank(pct=True)

# 예시
# # KOSPI 200 종목 내에서 PER 순위
# df = percentile_rank_normalization(df)
```

2.4. Z-Score 롤링 (Z-Score Rolling)

시계열 데이터의 평균과 표준편차를 롤링 윈도우 방식으로 계산하여 정규화한다. 이는 시계열 특성을 유지하면서 특정 기간 동안의 상대적 위치를 파악하는 데 적합하며, 모멘텀 지표{{3}}나 변동성 지표에 유용하다.

수학적 정의: $$ Z_{score}(t) = \frac{X_t - \mu_{\text{rolling}}(t)}{\sigma_{\text{rolling}}(t)} $$ 여기서, X_t는 시점 t의 데이터, $\mu_{\text{rolling}}(t)$와 $\sigma_{\text{rolling}}(t)$는 시점 t까지의 롤링 윈도우 내 평균과 표준편차이다.

Python 수도코드:

```python
def z_score_rolling_normalization(series, window=20):
    """
    롤링 Z-Score 정규화를 적용합니다.
    Args:
        series (pd.Series): 정규화할 데이터 시리즈.
        window (int): 롤링 윈도우 크기.
    Returns:
        pd.Series: 정규화된 데이터 시리즈.
    """
    rolling_mean = series.rolling(window=window).mean()
    rolling_std = series.rolling(window=window).std()
    # 표준편차가 0인 경우를 방지하기 위해 nan으로 처리하거나 작은 값 추가
    normalized_series = (series - rolling_mean) / rolling_std
    return normalized_series

# 예시
# df = z_score_rolling_normalization(df)
```

2.5. 지표별 정규화 매핑 테이블

지표명	정규화 방법	이유
PER, PBR	Min-Max Scaling	안정적이며 극단적인 값의 발생 빈도가 상대적으로 낮은 지표에 적합.
ROE, 매출 성장률 YoY	Min-Max Scaling	펀더멘털 지표의 본질적 특성을 유지하면서 0-1 범위로 스케일링.
이벤트(공시 유형별 CAR)	Percentile Rank	종목 간 상대적인 이벤트 효과를 비교하는 데 유리하며, 분포 가정이 불필요.
RSI14, MACD	Sigmoid 정규화	과매수/과매도 구간 및 극단값에 대한 민감도를 완화하고 부드러운 변환을 제공.
볼린저밴드 관련 지표	Z-Score 롤링	시계열 특성을 살려 특정 기간 내 상대적 위치를 파악하며, 밴드 이탈 강도 측정에 유용.

이격도	Z-Score 롤링	주가와 이동평균 간의 괴리율이 시간에 따라 변동하는 특성을 반영하여 상대적 과열/과냉을 분석.
MA 크로스 관련 지표	Sigmoid 정규화	추세 전환 신호의 강도를 부드럽게 표현하고, 극단적인 추세 변화에 대한 민감도를 조절.

3. 가중치 안정성 및 턴오버 제약

동적 가중치 시스템은 시장 변화에 빠르게 반응하여 잠재적 이점을 제공하지만, 과도한 가중치 변화는 잦은 거래로 이어져 거래비용{{4}}을 증가시키고 포트폴리오 턴오버율을 높일 수 있다. 효율적인 포트폴리오 관리를 위해서는 이러한 trade-off를 고려해야 한다.

3.1. 가중치 급변 방지를 위한 지수이동평균(EMA) 스무딩 공식

가중치의 급격한 변화를 완화하여 불필요한 거래를 줄이기 위해 지수이동평균(EMA) 스무딩을 적용할 수 있다. EMA는 최근 데이터에 더 큰 가중치를 부여하면서도 과거 데이터의 영향을 점진적으로 반영하여 가중치의 안정성을 확보한다.

EMA 스무딩 공식: $$ w_i^{EMA}(t) = \alpha \times w_i^{\text{new}}(t) + (1 - \alpha) \times w_i^{EMA}(t-1) $$ 여기서, $w_i^{EMA}(t)$는 시점 t에서의 스무딩된 가중치, $w_i^{\text{new}}(t)$는 시점 t에서 새롭게 산출된 가중치(예: ML 메타모델 예측 가중치), α는 스무딩 계수($0 < \alpha < 1$)이다. α 값이 클수록 최근 가중치에 더 큰 비중을 둔다.

Python 수도코드:

```python
def ema_smoothing(new_weights, previous_ema_weights, alpha):
    """
    지수이동평균(EMA)을 사용하여 가중치를 스무딩합니다.
    Args:
        new_weights (pd.Series): 현재 시점의 새로운 가중치.
        previous_ema_weights (pd.Series): 이전 시점의 EMA 스무딩 가중치.
        alpha (float): 스무딩 계수 (0 < alpha < 1).
    Returns:
        pd.Series: EMA 스무딩된 가중치.
    """
    if previous_ema_weights is None:
        return new_weights # 초기 값
    return alpha * new_weights + (1 - alpha) * previous_ema_weights

# 예시
# alpha_smoothing = 0.2 # 0.1 ~ 0.3 범위에서 백테스팅을 통해 최적화
# smoothed_weights = ema_smoothing(dynamic_weights_ml.iloc,
previous_day_smoothed_weights, alpha_smoothing)
```

3.2. 거래비용 감안 시 리밸런싱 필터

거래비용(편도 0.015%, 세금 포함 약 0.3%)은 스윙 트레이딩의 수익성에 큰 영향을 미치므로, 이를 최소화하기 위한 리밸런싱 필터가 필수적이다. 턴오버율 제한은 마르코위츠 모델의 한계로 지적되었던 거래비용 및 카디널리티 제약과 같은 비볼록(nonconvex) 문제를 해결하기 위한 중요한 고려 사항이다. 포트폴리오 리밸런싱 시 최적의 빈도와 임계값을 설정하는 것은 수익성과 위험을 모두 고려해야 한다.

- **가중치 변화 임계값 (Weight Change Threshold)**: 현재 보유 종목의 가중치가 특정 임계값(예: 1% 또는 2%) 이상 변동했을 때만 리밸런싱을 수행한다. 즉, $|w_i^{EMA}(t) - w_i^{\text{current}}(t-1)| > \text{threshold}$ 일 경우에만 거래를 실행한다.
- **최소 거래 금액 (Minimum Trade Size)**: 거래비용이 상대적으로 높은 소액 거래를 피하기 위해 최소 거래 금액을 설정한다. 계산된 포지션 사이즈가 특정 금액 미만일 경우 거래를 하지 않거나, 다음 리밸런싱 시점까지 대기한다.
- **턴오버율 제한 (Turnover Limit)**: 총 포트폴리오 턴오버율이 특정 비율(예: 일일 10%, 주간 50%)을 초과하지 않도록 제약 조건을 설정한다. 이는 포트폴리오 최적화 문제에 제약 조건으로 포함될 수 있다.
- **손실 회피 리밸런싱 (Loss Aversion Rebalancing)**: 손실 중인 포지션에 대해서는 리밸런싱을 유예하거나, 가중치를 점진적으로 줄이는 전략을 사용하여 손실 확정을 회피하고 잠재적 반등 기회를 모색할 수 있다.

Python 수도코드:

```python
def apply_rebalancing_filter(current_weights, target_weights,
holding_stocks,
                             weight_change_threshold=0.01,
min_trade_value=1000000,
                             max_daily_turnover_ratio=0.1,
current_portfolio_value=100000000):
    """
    가중치 안정성 및 턴오버 제약을 적용하여 실제 리밸런싱 주문을 필터링합니다.
    Args:
        current_weights (pd.Series): 현재 포트폴리오의 가중치.
        target_weights (pd.Series): ML 모델이 예측한 목표 가중치.
        holding_stocks (list): 현재 보유 중인 종목 리스트.
        weight_change_threshold (float): 가중치 변화 임계값.
        min_trade_value (float): 최소 거래 금액.
        max_daily_turnover_ratio (float): 일일 최대 턴오버 비율.
        current_portfolio_value (float): 현재 포트폴리오 총 가치.
    Returns:
        dict: {stock_code: trade_size} 형태의 최종 거래 주문.
    """
    trade_orders = {}
    total_trade_value = 0

    # 목표 가중치를 EMA 스무딩된 가중치로 사용한다고 가정
    # 실제로는 새로운 목표 가중치와 기존 가중치를 비교하여 거래 여부 결정

    # 1. 기존 보유 종목 처리
    for stock_code in holding_stocks.index:
        current_holding_weight = current_weights.get(stock_code, 0)
        target_holding_weight = target_weights.get(stock_code, 0)

        weight_diff = target_holding_weight - current_holding_weight

        if abs(weight_diff) > weight_change_threshold:
            # 예상 거래 금액 계산
            trade_value = abs(weight_diff) * current_portfolio_value

            if trade_value >= min_trade_value:
                # 턴오버 제약 확인 (초과시 비율 조정 필요)
                if (total_trade_value + trade_value) /
current_portfolio_value <= max_daily_turnover_ratio:
                    trade_orders = weight_diff # 양수: 매수, 음수: 매도
                    total_trade_value += trade_value
                else:
                    # 턴오버 초과 시, 추가 거래 제한 또는 비율 조정 로직 추가
                    pass

    # 2. 신규 매수 종목 처리
    for stock_code in target_weights.index:
        if stock_code not in holding_stocks.index: # 신규 종목
            target_new_weight = target_weights.get(stock_code, 0)
```

```
            if target_new_weight > weight_change_threshold: # 일정 가중치 이상일 때만 신규 매수
                trade_value = target_new_weight * current_portfolio_value

                if trade_value >= min_trade_value:
                    if (total_trade_value + trade_value) / current_portfolio_value <= max_daily_turnover_ratio:
                        trade_orders = target_new_weight
                        total_trade_value += trade_value
                    else:
                        pass

    # 최종적으로 trade_orders의 가중치 합이 1이 되도록 조정 (선택 사항)
    # 실제 구현에서는 금액으로 변환하여 주문을 생성
    return trade_orders

# 예시
# # target_weights_today는 ML 메타모델 EMA 스무딩 후의 최종 목표 가중치
# # current_portfolio_weights는 전일 포트폴리오 가중치
# # current_holding_stocks는 현재 보유 종목 (DataFrame 등)
# # final_trade_signals = apply_rebalancing_filter(current_portfolio_weights, target_weights_today, current_holding_stocks)
```

근거 논문 목록

- **Factor Models, Machine Learning, and Asset Pricing**
  - Stefana Giglio, Bryan Kelly, Dacheng Xiu, 2022, Annual Review of Financial Economics.
  - 팩터 모델과 머신러닝을 이용한 자산 가격 결정 이론의 방법론적 기여를 다루며, 예상 수익률, 팩터, 위험 노출, 위험 프리미엄 추정 등에 ML을 활용하는 방법을 제시한다.

- **Factor Model Comparisons with Conditioning Information**
  - Wayne E. Ferson, Andrew F. Siegel, Junbo L. Wang, 2024, Journal of Financial and Quantitative Analysis.
  - 지연된 정보를 활용하여 팩터 모델을 테스트하는 방법론을 개발하며, 동적으로 거래되는 포트폴리오의 최적 샤프 비율과 바이어스 조정에 대해 논한다.

- **Dynamic Portfolio Choice with Linear Rebalancing Rules**
  - Ciamac C. Moallemi, Mehmet Sağlam, 2017, Journal of Financial and Quantitative Analysis.
  - 거래비용, 거래 제약 및 위험 고려 사항을 포함하는 동적 포트폴리오 최적화 문제에 대한 선형 리밸런싱 규칙을 제안한다.

- **Developing a dynamic portfolio selection model with a self-adjusted rebalancing method**
  - J Jung, S Kim, 2017, Journal of the Operational Research Society.
  - 변화하는 시장 상황을 고려한 동적 포트폴리오 선택 모델(DPSM)과 자체 조정 리밸런싱(SAR) 방법을 제안한다.

- **Detecting bearish and bullish markets in financial time series using hierarchical hidden Markov models**
  - L Oelschläger, T Adam, 2023, Statistical Modelling.
  - 계층적 은닉 마르코프 모델(HMM)을 사용하여 금융 시계열 데이터에서 강세장과 약세장을 감지하는 방법을 제시한다.

- **Dynamic Weighting Multi Factor Stock Selection Strategy Based on XGboost Machine Learning Algorithm**
  - Li Jidong, Zhang Ran, 2018, 2018 IEEE International Conference of Safety Produce Informatization (IICSPI).
  - XGBoost 모델 기반의 동적 가중치 다중 팩터 주식 선정 전략을 제시하며, XGBoost가 IC 계수 예측에 효과적임을 실증한다.

- **A Machine Learning View on Momentum and Reversal Trading**
  - Zhixi Li, Vincent Tam, 2018, Algorithms.
  - 머신러닝 기법(DT, SVM, MLP, LSTM)을 사용하여 주식 시장의 모멘텀 및 평균회귀 효과를 분석하고 수익성 있는 거래 전략을 구축하는 방법을 탐구한다. 특히 시장 상태에 따른 전략 전환을 제안한다.

- **Long Memory of Financial Time Series and Hidden Markov Models with Time‐Varying Parameters**
  - Peter Nystrup, Henrik Madsen, Erik Lindström, 2016, Journal of Forecasting.
  - 시간 가변 파라미터를 갖는 HMM이 금융 시계열의 장기 기억 효과를 재현하고 예측 성능을 향상시킬 수 있음을 보여준다.

- **A Novel Dynamic Neural System for Nonconvex Portfolio Optimization With Cardinality Restrictions**
  - Xinwei Cao, Shuai Li, 2023, IEEE Transactions on Systems Man and Cybernetics Systems.
  - 거래비용 및 카디널리티 제약과 같은 비볼록 포트폴리오 최적화 문제를 해결하기 위한 동적 신경망 모델을 제안한다.

- **Out‐of‐sample volatility prediction: Rolling window, expanding window, or both?**
  - Y Feng, Y Zhang, Y Wang, 2024, Journal of Forecasting.
  - 롤링 윈도우와 확장 윈도우를 모두 사용하는 새로운 변동성 예측 접근 방식을 제안하며, 예측 가능성 모멘텀(MoP)의 존재를 입증한다.

- **Adaptive rolling window selection for minimum variance portfolio estimation based on reinforcement learning**
  - Bruno Gasperov, Fredi Šaric,´ Stjepan Begušic,´ Zvonko Kostanjcˇar, 2020, IEEE.
  - 강화 학습 기반의 모델 프리 정책을 사용하여 최소 분산 포트폴리오 추정에 최적 롤링 윈도우 크기를 적응적으로 결정하는 방법을 제시한다.

- **A Universal End-to-End Approach to Portfolio Optimization via Deep Learning**
  - Zhang Chao, Zhang Zihao, Cucuringu, Mihai, Zohren, Stefan, 2021, arXiv (Cornell University).
  - 딥러닝을 통해 자산 분포를 직접 도출하는 포트폴리오 최적화를 위한 보편적인 엔드-투-엔드 프레임워크를 제안하며, 카디널리티 및 개별 종목 최대 포지션과 같은 제약 조건을 고려한다.

- **A Three-phase Approach to an Enhanced Index-tracking Problem with Real-life Constraints**
  - O. Strub, S. Brandinu, D Lerch, J. Schaller, N. Trautmann, 2019, The Engineering Economist.
  - 실제 제약 조건을 포함하는 향상된 인덱스 트래킹 전략을 위한 포트폴리오 리밸런싱 문제를 다루며, 거래비용 후 벤치마크를 초과하는 누적 수익률을 달성한다.